

BOUNDLESS ENTERPRISES

CENTRAL PLATFORM / DIGITAL OPERATING SYSTEM

Building what comes after.

A unified corporate platform for the Boundless Enterprises holding company: public company discovery, subsidiary access, employee identity, onboarding, payments, shared services, and a private intelligence layer fed by the operating companies.

Architecture Single repository. Next.js frontend. .NET modular monolith API. SQL Server central data store.	Business Model Boundless Enterprises is the holding company. Each subsidiary keeps its own identity and operating systems.
Shared Platform Identity, employee access, onboarding, payments, documents, company routing, administration.	Intelligence Private cross-company analytics and event ingestion. Not exposed on the public site.

Technology for people. Enterprise for humanity.

PLATFORM DEFINITION

The Core Idea

Boundless Enterprises is the holding company. The central application is the holding company's digital operating layer, not the website for any one subsidiary. Boundless, Firefin, SkaffaldOS, DigitalHeavyWeights, and future companies remain distinct organizations underneath it.

The public side explains Boundless Enterprises and provides a deliberate path into its portfolio. Once a visitor chooses a company, that company page can expose the appropriate actions: visit the company, employee login, make a payment, contact, or other company-specific resources.

The authenticated side becomes shared corporate infrastructure. One identity can belong to multiple companies with different roles and permissions. The private intelligence layer then receives normalized data from the separate operating applications and provides executive-level visibility across the portfolio.

To help usher in a new age of human-centered enterprise - building technology, media, experiences, and institutions that connect people, expand human potential, and improve the world they inhabit.

Public Corporate Site Boundless Enterprises itself: mission, philosophy, about, careers, contact, and a path to Our Companies.	Company Directory A portfolio view where each subsidiary receives a distinct profile and routing page.
Enterprise Portal Employee identity, onboarding, documents, payments, permissions, and company access.	Enterprise Intelligence Internal-only financial, operational, customer, growth, and cross-company intelligence.

ORGANIZATIONAL MODEL

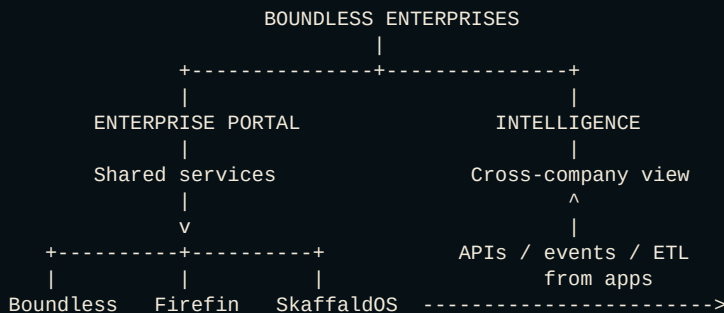
Holding Company Structure

```

BOUNDLESS ENTERPRISES
Holding Company
|
+-- Boundless
|   +-- Media / IP / Worldbuilding
|   |
+-- Firefin
|   +-- Food / Hospitality / Consumer Products
|   |
+-- SkaffaldOS
|   +-- Contractor SaaS
|   |
+-- DigitalHeavyWeights
|   +-- Technology / Software
|   |
+-- Future Companies
    +-- Independent brands and ventures
  
```

The central rule is simple: **the subsidiaries operate independently; the holding company sees across them**. Each company can retain its own domain, branding, customers, employees, application, database, and payment account while still participating in shared enterprise services.

The relationship should be expressed architecturally as:



VISUAL DIRECTION

Homepage



Homepage direction: Boundless Enterprises itself is the subject. The visual language combines corporate permanence, luxury restraint, modern interaction, and Boundless-style cosmic / occult geometry.

VISUAL DIRECTION

Our Companies

BOUNDLESS ENTERPRISES

About Philosophy Companies Careers Contact **Our Companies**

THE BOUNDLESS PORTFOLIO

OUR COMPANIES

Distinct companies. Shared philosophy.
Built for a more human future.

DIFFERENT COMPANIES. A BRIGHTER TOMORROW.

PEOPLE IDEAS COMPANIES A BRIGHTER TOMORROW.

FURTHER HUMAN TOGETHER.

A House of Independent Companies

Boundless Enterprises is a holding company built for the long horizon. Each company operates with its own identity, mission, and market, while sharing a common belief: technology, enterprise, and creativity should serve people first.

*Independent companies.
A more human tomorrow.*

Our Companies

DIFFERENT WORLDS. A SHARED PURPOSE.

MEDIA

Boundless

Media, worldbuilding, and enduring story systems.

We create stories, worlds, and media that expand perspective and bring people closer together.

[View Company →](#)

HOSPITALITY

Firefin

Food, hospitality, and consumer products shaped around experience.

We build exceptional food, beverage, and lifestyle brands that turn everyday moments into something more meaningful.

[View Company →](#)

SOFTWARE

SkaffaldOS

Software for contractors, operations, and business growth.

We build practical software that helps real people do real work and grow stronger businesses.

[View Company →](#)

TECHNOLOGY

DigitalHeavyWeights

Technology, software, and digital product development.

We design and build powerful digital products that solve real problems and create lasting value.

[View Company →](#)

FUTURE

Future Ventures

New companies built when the mission and timing are right.

We explore what's next, backing and building new ideas for a more human future.

[View Company →](#)

Shared Foundation

Long Horizons

We build for decades, not quarters.

Human-First Design

People are the center. Everything else is a tool.

Independent Missions

Each company keeps its own identity and purpose.

Real Value

Build things worth keeping.

Different companies. One horizon.

Explore the companies of Boundless Enterprises and enter the worlds they are building.

[Enter a Company](#)

BOUNDLESS ENTERPRISES

About Philosophy Companies Careers Contact

© Boundless Enterprises

A MORE HUMAN TOMORROW ALWAYS

Portfolio page direction: the holding company presents distinct companies under a shared philosophy, then routes the visitor into a specific subsidiary.

VISUAL DIRECTION

Company Page - Firefin

BOUNDLESS ENTERPRISES

[About](#)
[Philosophy](#)
[Companies](#)
[Careers](#)
[Contact](#)
[Our Companies](#)

Our Companies / Firefin

GOOD
FOOD
BRIGHTER
TOMORROWS

A BOUNDLESS ENTERPRISES COMPANY

Firefin

Food, hospitality, and consumer products built around experience.

We build meaningful food and hospitality experiences for people — bringing exceptional flavors, moments, and brands into everyday life.

PEOPLE
PLACES
GOOD FOOD
A BRIGHTER
TOMORROW

[Visit Company →](#)
[Employee Login](#)
[Make a Payment](#)
[Contact](#)

MORE
THAN
A MEAL

About Firefin

Firefin is a hospitality and consumer brand focused on memorable food, beverage, lifestyle, and guest experiences. We bring people together through thoughtful concepts, exceptional products, and a deep belief in the power of a shared table.

From everyday moments to special occasions, Firefin creates experiences that nourish people and inspire a more connected, vibrant world.

EXCEPTIONAL
FOOD
BRIGHTER
PEOPLE

A MORE
HUMAN
TABLE

What Firefin Does

Food & Hospitality

We create and operate restaurants, hospitality concepts, and culinary experiences that bring people together around exceptional food and service.

Consumer Products

We develop food, beverage, and lifestyle products that extend our hospitality philosophy into everyday life, at home and on the go.

Brand Experiences

We design immersive brand experiences, events, and spaces that connect people, culture, and communities through the joy of food and shared moments.

Company Access

Get to the resources, services, and information you need.

Visit Firefin.com
Explore menus, locations, products, and more. →

Employee Login
Access internal tools and resources. →

Make a Payment
Pay bills, invoices, or outstanding balances. →

Vendor / Partner Inquiry
Work with us. Share your information and proposal. →

At a Glance

Industry
Hospitality

Focus
Food, beverage, experience

Parent Company
Boundless Enterprises

Status
Active Venture

Aligned with Boundless

Firefin is proud to be part of a larger vision — different companies, a shared purpose.

Human-First
People are the center. Everything else is a tool.

Long Horizons
We build for decades, not quarters.

Craft & Experience
Extraordinary experiences create a brighter world.

Real Value
We make things that matter, for people's real lives.

BETTER
FOOD
BRIGHTER
TOMORROWS

Enter Firefin

Explore the brand, the experiences, and what's next.

[Visit Firefin.com →](#)

GOOD
PEOPLE
GREAT FOOD
BRIGHTER
TOMORROWS

BOUNDLESS ENTERPRISES

[About](#)
[Philosophy](#)
[Companies](#)
[Careers](#)
[Contact](#)

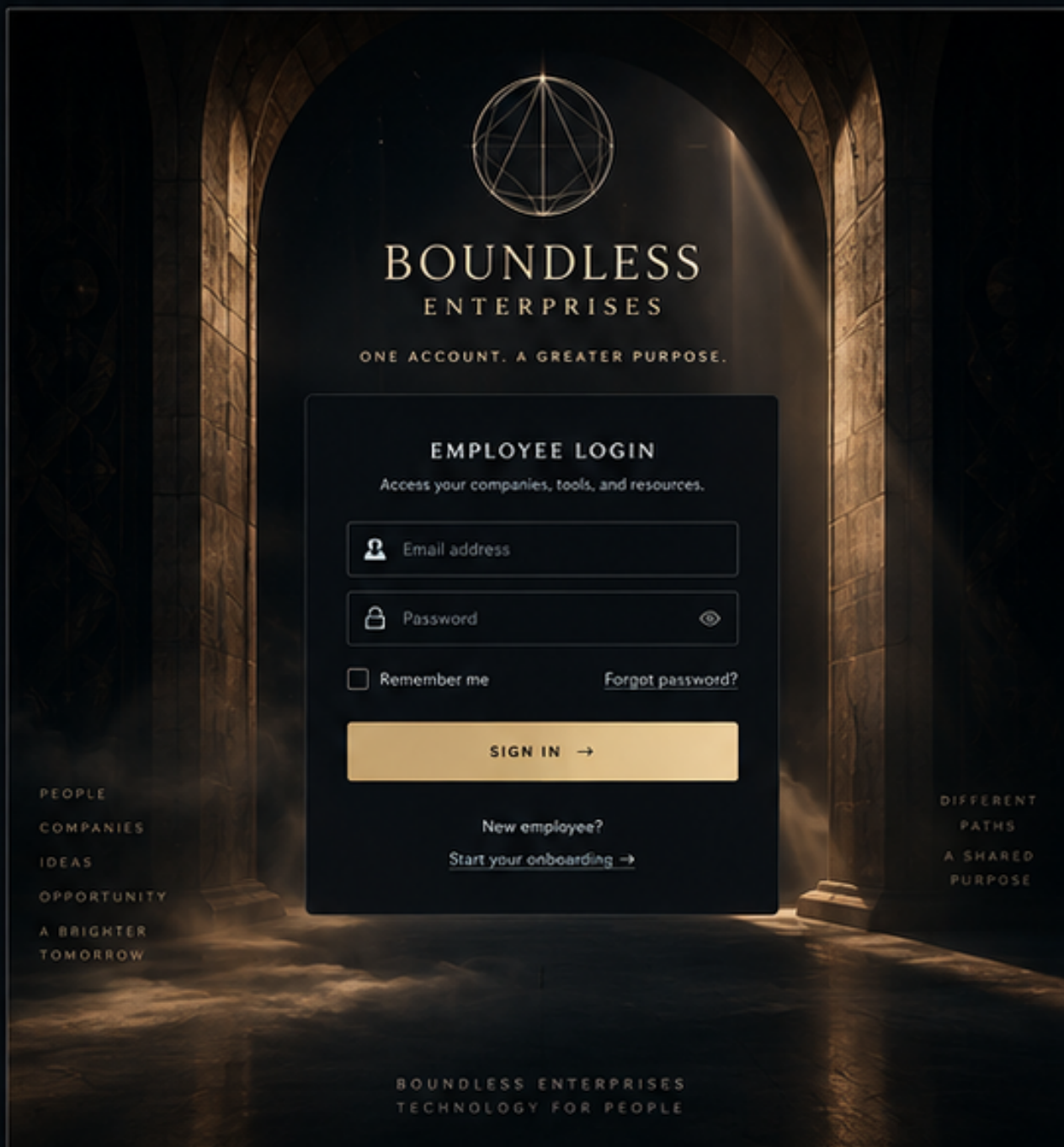
© Boundless Enterprises

A MORE
HUMAN
TOMORROW
ALWAYS

Example subsidiary page. This is where company-specific actions appear: visit the operating company, employee login, make a payment, contact, and related resources.

VISUAL DIRECTION

Employee Login



Employee access stays visually part of Boundless Enterprises while acting as the gateway into company-specific resources and permissions.

UX MODEL

Public Site Information Architecture

```
BOUNDLESS ENTERPRISES
|
|-- Home
|   |-- Who we are
|   |-- Why we exist
|   |-- Philosophy
|   |-- Long-horizon operating doctrine
|
|-- About
|
|-- Philosophy
|
|-- Our Companies
|   |
|   |-- Boundless
|   |   |-- About
|   |   |-- Visit Company
|   |   |-- Employee Login
|   |   |-- Payment
|   |
|   |-- Firefin
|   |   |-- About
|   |   |-- Visit Company
|   |   |-- Employee Login
|   |   |-- Payment
|   |
|   |-- SkaffaldOS
|   |   |-- About
|   |   |-- Visit Company
|   |   |-- Employee Login
|   |   |-- Payment
|
|-- Careers
|-- Contact
```

The Intelligence Platform does not appear in the public hierarchy. It is internal infrastructure behind authentication and authorization.

Homepage Distinct to Boundless Enterprises. It sells the holding-company mission and long-term philosophy, not individual products.	Our Companies Reveals the portfolio. Visitors intentionally choose which company they want to enter.
Company Page Explains the subsidiary and exposes practical actions such as Visit, Employee Login, Payment, Contact, or Partner Inquiry.	Portal Only after authentication do employees and executives see operational systems and intelligence.

FUNCTIONAL ARCHITECTURE

Shared Enterprise Services

Enterprise Directory Companies, descriptions, logos, domains, status, contact details, portfolio taxonomy, and public routing.	Identity and Access Central accounts, company memberships, roles, permissions, authentication, password recovery, and access boundaries.
Employees Employee records, company employment relationships, status, organizational access, and internal resources.	Onboarding Reusable onboarding engine with company-specific templates, steps, training, approvals, documents, and completion tracking.
Payments Company-aware invoices, one-time payments, subscriptions, balances, and routing to the correct payment configuration.	Documents Policies, agreements, onboarding documents, internal resources, and assignment to employees / companies.
Integrations Company app APIs, webhooks, event ingestion, configuration, sync jobs, and external providers.	Intelligence Metrics, events, snapshots, trends, alerts, cross-company comparisons, and future AI analysis.

One identity should be able to map to multiple companies without duplicate accounts:

```
User
+-- Boundless Enterprises account
  |
  +-- Boundless      -> Owner / Executive
  +-- Firefin        -> Owner / Executive
  +-- SkaffaldOS     -> Owner / Executive

Employee A
+-- Firefin          -> Restaurant Manager
+-- Boundless        -> No Access

Employee B
+-- Boundless        -> Writer / Ring assignment
+-- Firefin          -> No Access
```

REUSABLE ENGINES

Onboarding and Payments

Onboarding should be a reusable engine, not hard-coded pages. Each company selects or defines an onboarding template while the underlying process, status tracking, approvals, and document handling remain shared.

```
FIRE FIN - KITCHEN EMPLOYEE
1. Personal information
2. Employment documents
3. Tax / payroll information
4. Emergency contact
5. Policies
6. Food safety training
7. Firefin orientation
8. Equipment / access
9. Manager approval
```

```
BOUNDLESS - CREATIVE EMPLOYEE
1. Personal information
2. NDA / IP agreement
3. Core onboarding
4. Soul Skill
5. Ring assignment
6. Mask / Veil / Quill
7. Project access
8. Software accounts
9. First Horizon
```

Payments should also be company-aware. A customer may arrive through a specific company page, but the central platform can provide the shared payment experience while preserving company separation.

```
Payment request
|
v
CompanyId
|
+-- Firefin      -> Firefin payment configuration / ledger
+-- Boundless   -> Boundless payment configuration / ledger
+-- Skaffald0S  -> Skaffald0S payment configuration / ledger
```

CHOSEN STACK

Technical Direction

Frontend Next.js + React + TypeScript. App Router. Public, auth, and portal route groups.	Backend C# / .NET Web API. Modular monolith with clean internal boundaries.
Database Microsoft SQL Server. EF Core for application persistence and migrations.	Database Tooling SSMS is the management / inspection client for SQL Server; it is not the database itself.
Architecture One repository, one frontend, one API, one central holding-company database.	Company Systems Separate operating applications and databases communicate upward through APIs, webhooks, events, or controlled sync jobs.

The preferred starting architecture is a **modular monolith**. It provides strong boundaries without turning the codebase into a microservice maze before the operational scale justifies it.

FILE SYSTEM

Repository Root

```
BoundlessEnterprises/  
|  
+-- frontend/  
+-- backend/  
+-- docs/  
+-- scripts/  
+-- .github/  
+-- .editorconfig  
+-- .gitignore  
+-- README.md
```

Keep the platform in one repository. The organization comes from business modules and clear layer boundaries, not from creating a separate repository or deployable service for every capability.

NEXT.JS / REACT / TYPESCRIPT

Frontend Structure

```

frontend/
|
+-- public/
|   +-- brand/
|   |   +-- logos/
|   |   +-- icons/
|   |   +-- symbols/
|   +-- images/
|   |   +-- corporate/
|   |   +-- companies/
|   +-- fonts/
|
+-- src/
|   +-- app/
|   |   +-- (public)/
|   |   |   +-- layout.tsx
|   |   |   +-- page.tsx
|   |   |   +-- about/page.tsx
|   |   |   +-- philosophy/page.tsx
|   |   |   +-- companies/
|   |   |   |   +-- page.tsx
|   |   |   |   +-- [slug]/page.tsx
|   |   |   +-- careers/page.tsx
|   |   |   +-- contact/page.tsx
|   |   +-- (auth)/
|   |   |   +-- layout.tsx
|   |   |   +-- login/page.tsx
|   |   |   +-- forgot-password/page.tsx
|   |   |   +-- reset-password/page.tsx
|   |   +-- (portal)/
|   |   |   +-- portal/
|   |   |   |   +-- layout.tsx
|   |   |   |   +-- page.tsx
|   |   |   |   +-- companies/
|   |   |   |   +-- employees/
|   |   |   |   +-- onboarding/
|   |   |   |   +-- payments/
|   |   |   |   +-- documents/
|   |   |   |   +-- intelligence/
|   |   |   |   +-- integrations/
|   |   |   |   +-- settings/
|   |   +-- features/
|   |   |   +-- auth/
|   |   |   +-- companies/
|   |   |   +-- employees/
|   |   |   +-- onboarding/
|   |   |   +-- payments/
|   |   |   +-- documents/
|   |   |   +-- intelligence/
|   |   |   +-- integrations/
|   |   +-- components/
|   |   |   +-- ui/
|   |   |   +-- layout/
|   |   |   +-- shared/
|   |   +-- lib/
|   |   |   +-- api/
|   |   |   +-- auth/
|   |   |   +-- validation/
|   |   |   +-- constants/
|   |   |   +-- formatting/
|   |   |   +-- utils/
|   |   +-- hooks/
|   |   +-- types/
|   |   +-- styles/globals.css
|   +-- tests/
+-- .env.local.example
+-- eslint.config.mjs
+-- next.config.ts
+-- package.json
+-- tsconfig.json

```

Rule: app/ represents routing. features/ represents feature implementation. Route files should stay thin and compose feature-level components instead of becoming giant pages full of business logic.

CONSISTENCY RULE

Frontend Feature Anatomy

```
src/features/payments/  
+-- components/  
|   +-- PaymentsDashboard.tsx  
|   +-- InvoiceCard.tsx  
|   +-- PaymentForm.tsx  
+-- hooks/  
|   +-- useInvoices.ts  
|   +-- usePayments.ts  
+-- api/  
|   +-- payments.api.ts  
+-- schemas/  
|   +-- payment.schema.ts  
+-- types/  
    +-- payment.types.ts  
  
src/app/(portal)/portal/payments/page.tsx  
  -> route / layout responsibility only  
  -> renders PaymentsDashboard
```

Repeat the same internal shape when it adds value: components, hooks, API access, validation schemas, and types stay close to the feature they serve. Avoid a global dumping ground of unrelated models and helpers.

C# / .NET

Backend Solution

```
backend/
|
+-- BoundlessEnterprises.sln
|
+-- src/
|   +-- BoundlessEnterprises.Api/
|   +-- BoundlessEnterprises.Application/
|   +-- BoundlessEnterprises.Domain/
|   +-- BoundlessEnterprises.Infrastructure/
|
+-- tests/
|   +-- BoundlessEnterprises.UnitTests/
|   +-- BoundlessEnterprises.IntegrationTests/
```

Four production projects are enough. This preserves dependency boundaries without creating project soup. Domain contains business concepts, Application contains use cases, Infrastructure handles external technology, and Api exposes HTTP.

BUSINESS CONCEPTS

Domain Project

```
BoundlessEnterprises.Domain/  
|  
+-- Common/  
|   +-- Entity.cs  
|   +-- AuditableEntity.cs  
|   +-- DomainEvent.cs  
|  
+-- Companies/  
|   +-- Company.cs  
|   +-- CompanyStatus.cs  
|   +-- CompanyType.cs  
|  
+-- Employees/  
|   +-- Employee.cs  
|   +-- Employment.cs  
|   +-- EmployeeStatus.cs  
|  
+-- Identity/  
|   +-- Role.cs  
|   +-- Permission.cs  
|  
+-- Payments/  
|   +-- Invoice.cs  
|   +-- Payment.cs  
|   +-- Subscription.cs  
|   +-- PaymentStatus.cs  
|  
+-- Onboarding/  
|   +-- OnboardingProcess.cs  
|   +-- OnboardingTemplate.cs  
|   +-- OnboardingStep.cs  
|  
+-- Intelligence/  
|   +-- Metric.cs  
|   +-- BusinessEvent.cs  
|   +-- DataSource.cs  
|  
+-- Integrations/  
|   +-- Integration.cs
```

The Domain project should remain technology-agnostic: no controllers, no HTTP concerns, no Stripe implementation, and no EF Core persistence mechanics leaking into business behavior.

USE CASES

Application Project

```
BoundlessEnterprises.Application/  
|  
+-- Common/  
|   +-- Interfaces/  
|   +-- Exceptions/  
|   +-- Behaviors/  
|   +-- Models/  
|  
+-- Companies/  
|   +-- Commands/  
|       |   +-- CreateCompany/  
|       |       |   +-- CreateCompanyCommand.cs  
|       |       |   +-- CreateCompanyHandler.cs  
|       |       |   +-- CreateCompanyValidator.cs  
|       |   +-- UpdateCompany/  
|       |   +-- ArchiveCompany/  
|       +-- Queries/  
|           |   +-- GetCompany/  
|           |   +-- GetCompanies/  
|       +-- DTOs/  
|       +-- Validators/  
|  
+-- Employees/  
+-- Identity/  
+-- Payments/  
+-- Onboarding/  
+-- Documents/  
+-- Intelligence/  
+-- Integrations/
```

Organize by business capability first and by command / query inside the capability. Everything required for a use case should be easy to find together.

EXTERNAL TECHNOLOGY

Infrastructure and API

Infrastructure owns persistence and integration implementations:

```
BoundlessEnterprises.Infrastructure/  
|  
+-- Persistence/  
|   +-- ApplicationDbContext.cs  
|   +-- Configurations/  
|   +-- Migrations/  
|   +-- Seed/  
|  
+-- Identity/  
|   +-- IdentityService.cs  
|   +-- JwtService.cs  
|   +-- PasswordService.cs  
|  
+-- Payments/  
|   +-- Stripe/  
|       +-- StripePaymentService.cs  
|       +-- StripeWebhookHandler.cs  
|       +-- StripeOptions.cs  
|  
+-- Email/  
+-- Documents/  
|  
+-- Intelligence/  
|   +-- IntelligenceRepository.cs  
|   +-- EventIngestionService.cs  
|   +-- MetricAggregationService.cs  
|  
+-- Integrations/  
|   +-- Firefin/  
|       |   +-- FirefinClient.cs  
|       |   +-- FirefinOptions.cs  
|   +-- Boundless/  
|   +-- SkaffaldOS/  
|  
+-- DependencyInjection.cs
```

The API stays intentionally thin:

```
BoundlessEnterprises.Api/  
+-- Endpoints/  
|   +-- Auth/  
|   +-- Companies/  
|   +-- Employees/  
|   +-- Payments/  
|   +-- Onboarding/  
|   +-- Documents/  
|   +-- Intelligence/  
|   +-- Integrations/  
+-- Middleware/  
+-- Extensions/  
+-- Program.cs  
+-- appsettings.json  
+-- appsettings.Development.json
```

HTTP request -> authenticate / validate -> Application use case -> HTTP response

CENTRAL DATABASE

SQL Server Organization

Use SQL Server schemas to make the holding-company database readable in SSMS and to preserve capability boundaries at the database level.

```
BoundlessEnterprisesDb
|
+-- core
|   +-- Companies
|   +-- CompanySettings
|
+-- identity
|   +-- Users
|   +-- Roles
|   +-- Permissions
|   +-- UserCompanies
|   +-- UserRoles
|
+-- hr
|   +-- Employees
|   +-- Employments
|
+-- onboarding
|   +-- Templates
|   +-- Processes
|   +-- Steps
|   +-- EmployeeSteps
|
+-- billing
|   +-- Customers
|   +-- Invoices
|   +-- InvoiceItems
|   +-- Payments
|   +-- Subscriptions
|
+-- documents
|   +-- Documents
|   +-- DocumentAssignments
|
+-- integration
|   +-- Integrations
|   +-- DataSources
|   +-- WebhookEvents
|   +-- ImportJobs
|
+-- intelligence
|   +-- Metrics
|   +-- MetricValues
|   +-- BusinessEvents
|   +-- DailySnapshots
|
+-- audit
|   +-- AuditLogs
```

Example EF Core mapping:

```
builder.ToTable("Companies", "core");
```

CRITICAL RULE

Company Data Boundaries

The central holding-company database should not become a copy of every subsidiary's operating database. It stores the information Boundless Enterprises needs at the parent-company level.

```
Firefin Database
  |
  | API / events
  v
Boundless Enterprises Central Database
```

```
SkaffaldOS Database
  |
  | API / events
  v
Boundless Enterprises Central Database
```

Do not build this:

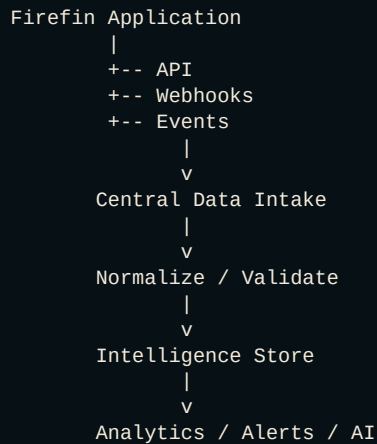
```
BoundlessEnterprisesDb
+-- FirefinOrders
+-- FirefinMenu
+-- FirefinRecipes
+-- SkaffaldProjects
+-- SkaffaldCRM
+-- BoundlessCharacters
+-- BoundlessLore
+-- ...
```

That structure collapses company boundaries and turns the holding company into the operational database for every subsidiary. The center should receive normalized enterprise data, not absorb every domain model.

PRIVATE LAYER

Enterprise Intelligence

The intelligence platform receives controlled data from the operating applications and converts it into a cross-company language of businesses, metrics, events, time, values, and dimensions.



Example normalized events:

```
OrderCompleted
{
  businessId: FIRE-001,
  orderId: 82919,
  revenue: 76.42,
  timestamp: ...
}
```

```
SubscriptionCreated
{
  businessId: SKAF-001,
  customerId: ...,
  tier: "Professional",
  mrr: 200,
  timestamp: ...
}
```

Do not have the intelligence UI constantly query every production database directly. Prefer explicit APIs, event publication, webhooks, or controlled ETL / sync jobs so company systems remain isolated and failures are contained.

FUTURE PRIVATE UX

Executive Command Center

BOUNDLESS ENTERPRISES - EXECUTIVE VIEW

Portfolio
 Financials
 Cash Flow
 Revenue
 Company Performance
 Employees
 Customers
 Risk
 Growth
 Capital Allocation
 Intelligence

Example question:
 "Why did total revenue fall last week?"

Possible response:
 Enterprise revenue decreased 8.7%.

Primary contributors:

SkaffaldOS	-\$8,400
Firefin	-\$2,900
Boundless	+\$1,200

SkaffaldOS accounted for most of the decline.
 New customer acquisition declined while churn remained stable.

This layer becomes more valuable as the portfolio grows because it creates one consistent executive surface without forcing the underlying companies to become the same business.

IMPLEMENTATION DOCTRINE

Architecture Rules to Protect

1. BusinessId Everywhere Every shared record that belongs to a company should have an explicit, enforceable company relationship. Never infer tenancy from UI state.	2. Thin Routes / Thin API Pages route and compose. Endpoints authenticate, validate, dispatch, and return. Core behavior lives in feature / application code.
3. Capability-First Folders Companies, Employees, Payments, Onboarding, Intelligence, and Integrations are first-class modules on both frontend and backend.	4. No Dumping Grounds Avoid giant Helpers, Managers, Services, Misc, CommonServices, Models, or Utils folders that erase domain ownership.
5. Subsidiary Independence A company can evolve its own application and database without forcing a central-platform rewrite.	6. Centralize Shared Work Identity, onboarding, common payments UX, documents, company directory, and cross-company intelligence belong in the holding-company platform.
7. Secure by Default Public corporate pages, company access, employee portal, and intelligence should be separate security surfaces with explicit authorization.	8. Start Modular, Split Later Begin as a modular monolith. Extract services only when scaling, ownership, deployment, or reliability pressure makes the boundary operationally useful.

BUILD TARGET

Final Repository Shape

```
BoundlessEnterprises/  
|  
+-- frontend/  
|   +-- public/  
|   +-- src/  
|       +-- app/  
|       +-- features/  
|       +-- components/  
|       +-- lib/  
|       +-- hooks/  
|       +-- types/  
|       +-- styles/  
|   +-- tests/  
|  
+-- backend/  
|   +-- BoundlessEnterprises.sln  
|   +-- src/  
|       +-- BoundlessEnterprises.Api/  
|       +-- BoundlessEnterprises.Application/  
|       +-- BoundlessEnterprises.Domain/  
|       +-- BoundlessEnterprises.Infrastructure/  
|   +-- tests/  
|       +-- BoundlessEnterprises.UnitTests/  
|       +-- BoundlessEnterprises.IntegrationTests/  
|  
+-- docs/  
|   +-- architecture/  
|   +-- database/  
|   +-- api/  
|   +-- security/  
|   +-- integrations/  
|  
+-- scripts/  
|   +-- development/  
|   +-- database/  
|  
+-- .github/  
|   +-- workflows/  
|  
+-- .editorconfig  
+-- .gitignore  
+-- README.md
```

*The world is not finished.
Neither is what humanity can build.*

BOUNDLESS ENTERPRISES